Fast Flow Engine

FFENGINE

Teknik Analiz Dokümanı (TAD)

Community: Query-Centric Batch ETL | Enterprise: Queue-Aware High-Performance Engine

Versiyon
TAD v6.0
Dönem
2026-Q1
Doküman Tipi
TAD — Teknik Analiz Dokümanı
Platform
ffengine Community + Enterprise
Orkestrasyon
Apache Airflow 3.1.6

İçindekiler

00 — Özet & Stratejik Amaç

ffengine (Fast Flow Engine), finansal sistemler, bankacılık ve kurumsal veri ambarları için tasarlanmış, veritabanı bağımsız (database-agnostic) bir yüksek performanslı veri entegrasyon platformudur. PostgreSQL, MSSQL ve Oracle arasında çift yönlü veri aktarımını; kod yazımını minimize eden YAML tabanlı ETL Studio arayüzü ve Apache Airflow entegrasyonu ile gerçekleştirir.

Ürün, değiştirilebilir Engine mimarisi üzerine kuruludur. ffengine community (önce çıkar) standart Python Engine ile tüm desteklenen veritabanları arasında çift yönlü aktarım sağlayan Airflow-native, low-code ETL deneyimi sunar — kullanıcılar ekran üzerinden kod yazmadan DAG oluşturup aktarımlarını gerçekleştirir. ffengine enterprise (sonra çıkar) aynı UI ve altyapı üzerinde Python Engine yerine C Engine'i devreye alarak queue-centric runtime, DB'ye özel yüksek hızlı aktarımlar ve gelişmiş akış kontrolü sağlar.

> **Temel Fark:** ffengine community, standart DBAPI (fetchmany/executemany) ile veritabanı bağımsız, kararlı ETL sunar. ffengine enterprise ise Binary Streaming (COPY, BCP, OCI_BATCH) ve Zero-Copy C Engine ile maksimum throughput sağlar. 'Ham hızda taşıma' Enterprise'a özgü bir yetenektir.

Kritik Performans Göstergeleri

< 500MB	100K+	< 3 dk	5–10x
Sabit RAM Kullanımı	Satır/Saniye Throughput	10M+ Satır Aktarımı	Enterprise Hız Faktörü

Stratejik Hedefler

- Performans & Ölçeklenebilirlik: 10M+ satır aktarımında OOM hatalarının eliminasyonu; Enterprise'da Binary Streaming ile maksimum throughput
- Veritabanı Bağımsızlığı: PostgreSQL, MSSQL, Oracle için tek çekirdek kod tabanı
- Low-Code ETL: YAML konfigürasyonu ile pipeline kurulumunu saatlerden dakikalara indirme
- Maliyet Tasarrufu: Tahminen %40 altyapı maliyeti azaltımı, bakım yükünün minimizasyonu

01 — Ürün Vizyonu & İki Çekirdekli Model (Engine-Swap)

ffengine, Apache Airflow'un orkestrasyon gücünü yüksek performanslı streaming mimarisi ile birleştirerek heterojen veritabanı ekosistemlerinde veri aktarımını standardize eden bir ETL motoru olmayı hedefler. Ürün, değiştirilebilir Engine mimarisi üzerine kuruludur: ETL Studio UI, Dialect/Agnostic katmanı, DB bağlantı yapıları ve Airflow entegrasyonu ortaktır; yalnızca veri taşıma motoru (Engine) değişir.

Community vs Enterprise Karşılaştırması

Özellik	ffengine community	ffengine enterprise
Mimari Yaklaşım	Query-Centric: fetchmany ile sorgula, executemany ile yaz — standart DBAPI döngüsü	Queue-Aware / Capability-First: Ingress/Egress Queue ile çift taraflı kontrollü veri akışı
Dil / Çekirdek	Python 3.12 / Standard DBAPI 2.0	C / C++ — Low-Level Native API
Aktarım Metodu	fetchmany + executemany (standart DBAPI batch)	Zero-Copy Streaming (C buffer'ları) + native bulk API
Performans	Referans (1x)	Yüksek Hız (5x – 10x+)
Orkestrasyon	Apache Airflow Operatörü	Apache Airflow Operatörü
Konfigürasyon	YAML / ETL Studio	YAML / ETL Studio
Lisans / Destek	Open Source / Topluluk	7/24 Kurumsal SLA & Destek
Dağıtım	PyPI / Docker Hub	Binary Wheels / Private Registry
Bağlantı Sürücüsü	psycopg3, cx_Oracle, pyodbc	Native OCI, libpq, SQL Server Native Client
Optimizasyon	—	SIMD talimat setleri, işlemci seviyesi paralelizasyon
Queue Runtime	Yok — klasik Streamer (generator pattern)	Ingress/Egress Queue, FFEnvelope, Ack/Nack, Checkpoint, Delivery Policy, Multi-Lane Pipeline
DB'ye Özel Hızlı Aktarım	Yok — executemany (standart DBAPI)	PostgreSQL Binary COPY, MSSQL BCP, Oracle OCI_BATCH
Batch Stratejisi	Sabit batch_size (parametrik, YAML'dan ayarlanır)	Adaptif mikro-batch (satır + byte + süre + hedef sistem feedback)
Delivery Guarantee	Best-effort; hata anında chunk rollback	At-Least-Once (varsayılan); Exactly-Once (koşullu — bkz. Delivery Guarantee Matrix)
Idempotency / Ordering	Yok — load_method (truncate/append/upsert) ile yönetilir	Ack/Nack contract + ordering_key + idempotent sink kombinasyonu ile sağlanır
DLQ / Checkpoint	Yok	Dead Letter Queue + PK/Offset bazlı Checkpoint & Resume

Engine-Swap Mimarisi — Hangi Katman Değişir?

Katman	Değişir Mi?	Açıklama
ETL Studio UI	Hayır — Ortak	Şema keşfi, DAG oluşturma/güncelleme, tag yönetimi, timeline — her iki versiyonda aynı
Dialect / Agnostic Layer	Hayır — Ortak	PostgresDialect, OracleDialect, MSSQLDialect, TypeMapper — her iki versiyonda aynı
DB Bağlantı Yapıları	Hayır — Ortak	DBSession, connection config, Airflow Connection ID'leri — aynı
YAML Konfigürasyon	Hayır — Ortak	Aynı YAML formatı, aynı parametreler — kod değiştirmeden Engine swap yapılabilir
Airflow Entegrasyonu	Hayır — Ortak	FFEngineOperator, Auto-DAG, Dynamic Task Mapping, XCom — her iki versiyonda aynı
Veri Taşıma Motoru	EVET — Değişir	Community: Python Engine (fetchmany + executemany, standart DBAPI) → Enterprise: C Engine (queue-centric runtime, Ingress/Egress Queue, native bulk API, checkpoint, delivery policy)

02 — Mimari Genel Bakış (Engine-Swap Dual-Core Architecture)

ffengine community — Standart DB Aktarım Akışı

SOURCE DB → EXTRACTOR (fetchmany cursor) → BUFFER ENGINE (Generator/Backpressure) → TRANSFORMER (Type Mapping/DDL) → LOADER (executemany standart) → TARGET DB

Community Tasarım Felsefesi: Tüm desteklenen veritabanları arasında çift yönlü veri aktarımı. Standart DBAPI (fetchmany ile oku, executemany ile yaz) kullanılır. DB'ye özel hızlı aktarım API'leri (binary COPY, BCP, OCI_BATCH) bu katmanda yoktur — bunlar Enterprise'a aittir.

ffengine enterprise — Queue-Centric Streaming Akışı

SOURCE DB → SOURCE PRODUCER (Queue-Aware Reader) → INGRESS QUEUE (Backpressure/Envelope) → TRANSFORMER (Type Mapping/DDL) → EGRESS QUEUE (Delivery/Checkpoint) → TARGET DB (Controlled Writer)

Enterprise Tasarım Felsefesi (Queue-Centric): Klasik 'Extractor → Loader' boru hattı yerine, veri iki taraflı queue bridge üzerinden akar. Kaynak tarafta Source Producer küçük paketler üretir ve Ingress Queue'ya bırakır; hedef tarafta Egress Queue, Target Consumer'a kontrollü commit ile veri teslim eder. DB'ye özel hızlı aktarımlar ve tam queue semantiği (checkpoint, ack/nack, delivery policy) bu katmanda sağlanır.

Temel Bileşenler — Community vs Enterprise

Bileşen	Community	Enterprise
Kaynak Okuyucu	Extractor — fetchmany ile server-side cursor üzerinden standart paketli okuma	Source Producer — Queue-aware reader; COPY TO STDOUT / native API ile küçük paketler üretip Ingress Queue'ya bırakır
Akış Motoru	Stream Engine — Python generator (yield) ile producer-consumer pattern; basit backpressure	Ingress Queue — Thread-safe kuyruk; FFEnvelope, checkpoint, delivery policy ile tam queue semantiği
Dönüştürücü	Transformer / TypeMapper — On-the-fly tip dönüşümü, DDL üretimi, TABLE_RULES (her iki versiyon ortak)	← Aynı (ortak katman)
Hedef Yazıcı	Loader — executemany ile standart batch yazma (standart DBAPI, tüm DB'ler)	Egress Queue + Target Consumer — Kuyruk üzerinden kontrollü mikro-batch commit; native bulk API (COPY FROM STDIN, BCP, OCI_BATCH) — Enterprise Only
Metadata Inspector	Kaynak şema keşfi, kolon/tip/nullable tespiti, DDL otomasyonu (ortak)	← Aynı (ortak katman)
Orkestrasyon	Apache Airflow — FFEngineOperator, XCom,	← Aynı (ortak katman)

Bileşen	Community	Enterprise
	Dynamic Task Mapping, Pools (ortak)	

03 — Katmanlı Mimari Yapısı

Sistem, birbirine gevşek bağlı (decoupled) katmanlardan oluşur. Community versiyonu klasik Extractor → Stream Engine → Loader operasyon katmanını kullanırken, Enterprise versiyonu aynı katmanı Source Producer → Ingress Queue → Egress Queue → Target Consumer şeklinde queue-centric olarak genişletir.

Beş Ana Katman

Katman	Görev	Modül / Teknoloji
1 — Orkestrasyon	Görev planlama, izleme, DAG yönetimi	Apache Airflow 3.1.6 / FFEngineOperator
2 — Yönetim (ETLManager)	Konfigürasyon okuma, strateji seçimi, loglama	ETLManager / Parser / Validator
3 — Soyutlama (Dialect)	DB-specific SQL sentakslarını kapsülleme	PostgresDialect / OracleDialect / MSSQLDialect
4 — Operasyon	Extract / Transform / Load. Community: Extractor→Stream→Loader. Enterprise: SourceProducer→IngressQueue→EgressQueue→TargetConsumer	Community: SourceReader · Streamer · TargetWriter | Enterprise: SourceAdapter · IngressQueue · EgressQueue · TargetConsumer · CheckpointStore
5 — Sürücü	Fiziksel DB bağlantısı	psycopg3 · python-oracledb · pyodbc · libpq · OCI

Bileşen Şeması (Teknik Katman Tablosu)

Katman	Görev	Teknoloji / Yaklaşım	Modül Adı
Ingestion Layer	Veriyi kaynaktan çekme	Community: Cursor-based fetchmany | Enterprise: Source Producer + COPY TO STDOUT + Queue-Aware Adapter	Community: SourceReader | Enterprise: SourceAdapter, SourceProducer
Stream / Queue Layer	Paketleme, buffer ve akış yönetimi	Community: Python Generators, yield pattern | Enterprise: Ingress Queue + Egress Queue, Envelope, Checkpoint, Delivery Policy	Community: Streamer | Enterprise: IngressQueue, EgressQueue, CheckpointStore
Mapping Engine	Tip dönüşümü ve DDL üretimi	Centralized TypeMapper (YAML bazlı)	TypeMapper, DDLGenerator
Delivery Layer	Hedefe veri yazma	Community: executemany (standart DBAPI) | Enterprise: Egress Queue → Target Consumer → Native Bulk API ile kontrollü mikro-batch commit	Community: TargetWriter | Enterprise: EgressQueue, TargetConsumer, DeliveryManager
Orchestration	Görev planlama ve	Apache Airflow Operator,	FFEngineOperat

Katman	Görev	Teknoloji / Yaklaşım	Modül Adı
	izleme	XCom	or
Dialect Manager	DB-specific SQL üretimi	BaseDialect → Postgres/Oracle/MSSQL	DialectManager
Metadata Inspector	Şema keşfi ve DDL otomasyonu	Native metadata API'ları (ALL_TAB_COLUMNS, information_schema)	MetadataInspector

04 — Veri Akış Protokolü (Streaming I/O)

ffengine community, query-centric yaklaşımla server-side cursor (fetchmany) ile okur, executemany ile yazar. Native bulk API yoktur; serileştirme yükü mevcuttur ancak standart DBAPI döngüsü ile yönetilebilir düzeyde tutulur. ffengine enterprise, queue-aware mimarisi sayesinde binary COPY, BCP ve Zero-Copy C Engine ile serileştirme yükünü tamamen ortadan kaldırır. Aşağıdaki tabloda hangi mekanizmanın hangi versiyona ait olduğu açıkça belirtilmiştir.

Streaming & Bulk Mekanizmaları — Versiyon Ayrımı

Mekanizma	Açıklama	Veritabanı	Performans Etkisi	Versiyon
Server-Side Cursor (fetchmany)	fetchmany(10000) ile kademeli, bellek dostu okuma. Standart DBAPI.	Tümü	Sabit RAM (OOM engelleme); serileştirme overhead'i devam eder	Community + Enterprise
executemany (standart)	Standart DBAPI batch yazma. Hedef DB'ye kayıt gönderir.	Tümü	Referans hız (1x)	Community + Enterprise
COPY TO STDOUT (Binary)	psycopg3 ile binary format COPY komutu — Python nesnesine dönüştürmeden ham binary bloklar. Subquery + partition desteği.	PostgreSQL (kaynak)	fetchmany'ye göre ~2-3x hızlı okuma; Python object overhead sıfır	Enterprise Only
COPY FROM STDIN (Binary)	psycopg3 ile binary format COPY komutu — en hızlı PG yükleme yöntemi.	PostgreSQL (hedef)	%30–40 düşük CPU; en hızlı yükleme	Enterprise Only
BCP / fast_executemany	pyodbc fast_executemany=True veya BCP protokolü ile yüksek hızlı MSSQL yükleme.	MSSQL	Standart executemany'ye göre ~10x hız	Enterprise Only
OCI_BATCH_ERRORS	Oracle native batch insert + hata toleransı ile yüksek hızlı yükleme.	Oracle	Hatalı satırı izole ederek bulk devam eder	Enterprise Only
Zero-Copy Streaming	Python nesnesine dönüştürmeden C buffer'ları üzerinden — sıfır-dönüşüm aktarım.	Tümü	5x–10x community'ye göre hız artışı	Enterprise Only
Backpressure Yönetimi	Hedef yazma hızına göre kaynak okuma hızı otomatik denge — RAM taşmalarını önler.	Tümü	RAM taşmalarının engellenmesi	Community + Enterprise

Streaming Protocol Pseudocode — yield generator pattern

```
class Streamer:
    def stream(self, source_cursor, target_writer, batch_size=10000):
        # Producer: Server-side cursor üzerinden chunk bazlı okuma
```

```
        def producer():
            while True:
                chunk = source_cursor.fetchmany(batch_size)
                if not chunk: break
                yield chunk  # belleğe biriktirmeden doğrudan pipe

        # Community: Her chunk'ı executemany (standart DBAPI) ile hedefe yazar
        for chunk in producer():
            typed_chunk = self.type_mapper.cast(chunk)  # on-the-fly type
casting
            target_writer.bulk_write(typed_chunk)        # Community:
executemany | Enterprise: COPY / BCP / OCI_BATCH
            self._apply_backpressure()

    def _apply_backpressure(self):
        if self.buffer.qsize() >= self.pipe_queue_max:
            time.sleep(0.01)  # throttle
```

PostgreSQL Binary COPY Queue Pipeline — Enterprise Only (PG → Queue → PG)

> **Enterprise Only:** Bu bölümdeki COPY TO STDOUT / COPY FROM STDIN kullanımı yalnızca Enterprise versiyonuna aittir. Community'de PG→PG aktarımı da standart fetchmany + executemany ile gerçekleştirilir.

PostgreSQL kaynak olduğunda (Enterprise), veri COPY (subquery) TO STDOUT (FORMAT BINARY) ile doğrudan binary bloklar halinde okunabilir. Bu bloklar Python nesnesine dönüştürülmeden thread-safe bir kuyruğa aktarılır ve hedef tarafta COPY FROM STDIN (FORMAT BINARY) ile yazılır.

```
import threading
from queue import Queue
import psycopg

# reader_workers ve writer_workers YAML config'den alınır
READER_WORKERS = config.get("reader_workers", 3)
WRITER_WORKERS = config.get("writer_workers", 5)
SENTINEL = None

pipe_queue = Queue(maxsize=config.get("pipe_queue_max", 8))

def reader_worker(partition_sql: str, source_dsn: str):
    """COPY TO STDOUT ile kaynak PG'den binary blok okur, kuyruğa bırakır"""
    with psycopg.connect(source_dsn) as conn:
        with conn.cursor().copy(
            f"COPY ({partition_sql}) TO STDOUT (FORMAT BINARY)"
        ) as copy:
            for data in copy:
                pipe_queue.put(data)  # binary blok → kuyruğa (backpressure
otomatik)
    pipe_queue.put(SENTINEL)

def writer_worker(target_dsn: str, target_table: str, columns: list):
    """Kuyruktan binary blok alır, COPY FROM STDIN ile hedefe yazar"""
    with psycopg.connect(target_dsn) as conn:
        col_list = ",".join(columns)
        with conn.cursor().copy(
            f"COPY {target_table} ({col_list}) FROM STDIN (FORMAT BINARY)"
        ) as copy:
            while True:
                data = pipe_queue.get()
                if data is SENTINEL:
                    pipe_queue.put(SENTINEL)  # diğer writer'lara yayılım
                    break
```

```
                copy.write(data)  # binary blok → hedef DB (Python obj dönüşümü
YOK)

# Partition'lı Paralel Başlatma
partition_sqls = partitioner.generate(table, READER_WORKERS)
readers = [threading.Thread(target=reader_worker, args=(sql, SRC_DSN)) for sql
in partition_sqls]
writers = [threading.Thread(target=writer_worker, args=(TGT_DSN, tbl, cols)) for
_ in range(WRITER_WORKERS)]
for t in readers + writers: t.start()
for t in readers + writers: t.join()
```

Asimetrik Parametrik Worker Mimarisi (N Reader / M Writer)

Parametre	Tip	Varsayılan	Açıklama
reader_workers	int	3	Paralel okuyucu worker sayısı. Her worker kendi partition aralığını okur. Enterprise'da COPY TO STDOUT, Community'de fetchmany kullanılır.
writer_workers	int	5	Paralel yazıcı worker sayısı. Enterprise'da COPY FROM STDIN / BCP / OCI ile yazar; Community'de executemany kullanılır.
pipe_queue_max	int	8	Reader ve writer arasındaki kuyruk derinliği. Backpressure kontrol mekanizması.
extraction_method	enum	auto	copy_binary: COPY TO STDOUT (Enterprise Only, PG→PG hızlı mod) | cursor: fetchmany (Community + Enterprise, cross-DB uyumlu) | auto: kaynağa göre otomatik seçim

Senaryo	Reader	Writer	Gerekçe
Hızlı kaynak, yavaş hedef (PG → MSSQL)	2	5	Kaynakta az bağlantı açılır, hedefte çoklu paralel yazıcı ile yavaş hedef kompanse edilir
Yavaş kaynak, hızlı hedef (Oracle → PG)	5	2	Kaynaktan paralel okuma ile veri çekme darboğazı giderilir. Enterprise: PG hedefte COPY FROM STDIN; Community: executemany
PG → PG (aynı hızda)	3	3	Community: fetchmany + executemany. Enterprise: COPY TO STDOUT → Queue → COPY FROM STDIN uçtan uca binary akış
Büyük tablo, DWH yükleme	4	8	Yazma tarafında yoğun commit baskısını dağıtmak için writer çoğaltılır

Partitioning Stratejileri

Mod	Açıklama	Kullanım Senaryosu	Sınıf
auto	Otomatik partition tespiti → kolon tipine göre alt stratejiye yönlendirilir	Genel kullanım	AutoNumericPartition

Mod	Açıklama	Kullanım Senaryosu	Sınıf
percentile	Değer dağılımına göre eşit bölümleme (percentile hesaplama)	Büyük, dağınık veri setleri	PercentileRangePartition
hash_mod	Hash modulo ile deterministic bölümleme	Uniform dağılımlı kolonlar	HashModuloPartition
explicit	Kullanıcı tanımlı sınırlar (min-max aralıkları)	Tarih/değer bazlı partition	ExplicitRangePartition
distinct	Kolondaki benzersiz değerlere göre bölümleme	Kategori/enum kolonları	DistinctValuePartition
full_scan	Partition uygulanmaz, tek parça olarak tam tablo okunur	Küçük tablolar / WHERE ile sınırlı	_FullScanStrategy

4B — Queue-Centric Mimari (Enterprise Only)

Kapsam: Bu bölümdeki tüm kavramlar (Ingress/Egress Queue, FFEnvelope, Ack/Nack, Delivery Policy, Multi-Lane Pipeline, Adaptif Mikro-Batch) Enterprise versiyonuna aittir. Community versiyonu klasik producer-consumer generator pattern ile çalışır.

ffengine enterprise, klasik streaming'in ötesinde, iki tarafı da queue semantiğiyle yöneten, küçük paketli, çoklu paralel akış çalıştırabilen bir Queue-Centric Data Integration Engine'dir. Bu yaklaşım, klasik ETL'den farklı olarak flow control, backpressure, retry, ack/nack, checkpoint, DLQ ve multi-lane concurrency gibi mesajlaşma kavramlarını veri entegrasyonuna getirir.

Hedef DBMS'e Queue Tabanlı Yazma Gerekçesi

Problem	Açıklama	Queue ile Çözüm
Uzun Transaction	Büyük batch'ler transaction süresini uzatır	Küçük paketler ile transaction süresi minimize edilir
Lock Süresinin Büyümesi	Tabloda geniş kapsamlı kilitler oluşur	Mikro-batch'ler lock kapsamını daraltır
WAL / Redo / Transaction Log Baskısı	Devasa log kayıtları oluşur	Küçük commit'lerle log baskısı dağıtılır
Replication Gecikmesi	Replikasyon slave'leri geride kalır	Kontrollü yazma ile replikasyon dengede tutulur
Connection & I/O Sıçramaları	Ani yük artışları sistem kararsızlığına yol açar	Sabit throughput ile I/O dengeli kalır
Deadlock Riski	Eş zamanlı büyük batch'ler deadlock üretir	Küçük paketler deadlock olasılığını minimize eder

FFEngine Core Nesneleri (Queue Runtime)

Nesne	Açıklama	Anahtar Kavramlar
FFEnvelope	Her veri paketi için standart sarmalayıcı. Header (metadata, partition key, sequence) ve payload (data batch) içerir.	envelope · header · payload · partition-key
Ack/Nack Contract	Her paket teslimi sonrası onay (ack) veya ret (nack) mekanizması. Exactly-once delivery yalnızca belirli koşullarda mümkündür (bkz. Delivery Guarantee Matrix); aksi hâlde at-least-once garanti edilir.	ack · nack · at-least-once · exactly-once (koşullu)
Retry & DLQ Policy	Hatalı paketler için exponential backoff ile yeniden deneme ve maksimum deneme sonrası Dead Letter Queue'ya yönlendirme.	retry · exponential-backoff · poison-message · DLQ
Checkpoint & Offset	Source offset ve target write ack takibi ile kaldığı yerden devam (resume) yeteneği.	checkpoint · offset · resume · idempotent-write

Nesne	Açıklama	Anahtar Kavramlar
Adaptive Batch Policy	Satır sayısı, byte limiti, transaction süresi ve hedef sistem yanıt durumuna göre adaptif küçültme.	row-limit · byte-limit · time-limit · adaptive
Delivery Policy	Paket teslimat stratejisi: at-least-once (varsayılan) veya exactly-once (koşullu — idempotent sink + checkpoint + ordering key kombinasyonu gerektirir). Hangi kombinasyonda hangi semantiğin sağlandığı Delivery Guarantee Matrix'te tanımlanmıştır.	ordered · at-least-once · exactly-once (koşullu) · configurable

Delivery Guarantee Matrix — Enterprise

Exactly-once semantiği FFEngine'de koşulludur; sağlanabilmesi için hedef sistemin idempotent write desteklemesi, checkpoint mekanizmasının aktif olması ve (sıra kritikse) ordering key tanımlanmış olması gerekir. Aşağıdaki matris, çalışma zamanı kombinasyonuna göre hangi delivery semantiğinin geçerli olduğunu tanımlar.

> **Temel Prensip:** Exactly-once FFEngine tarafından garanti edilmez; altyapı koşullarının bütünü sağlandığında ortaya çıkan bir sonuçtur. FFEngine at-least-once'ı her zaman garanti eder; exactly-once elde etmek için hedef sistemin idempotency desteği zorunludur.

Kombinasyon	Aktif Bileşenler	Delivery Semantiği	Notlar
Queue-backed + Idempotent Sink + Ordering Key	Ingress/Egress Queue + CheckpointStore + ordering_key tanımlı + hedef tablo PK/UNIQUE constraint	Exactly-Once (Koşullu)	Hedef sink'in idempotency'si (ör: UPSERT / INSERT … ON CONFLICT DO NOTHING) sayesinde yeniden teslimde duplicate oluşmaz. Ordering key lane içi sırayı garanti eder.
Queue-backed + Idempotent Sink (Ordering Key YOK)	Ingress/Egress Queue + CheckpointStore + hedef tablo PK/UNIQUE constraint	At-Least-Once + No-Duplicate (sırasız)	Duplicate yazılmaz ancak lane içi satır sırası garanti edilmez. Sıra önemsizse yeterlidir.
Native Queue (Oracle AQ / MSSQL Service Broker)	Native queue semantiği + FFEngine Ack/Nack contract + hedef idempotent write	Exactly-Once (Koşullu)	Native queue kendi delivery guarantee'sini sağlar; FFEngine ack/nack ile bunu hedef yazma katmanına yansıtır. Hedef idempotency

Kombinasyon	Aktif Bileşenler	Delivery Semantiği	Notlar
			zorunludur.
Native Queue + Hedef İdempotency YOK	Native queue + FFEngine Ack/Nack (hedef tablo UNIQUE/PK yok)	At-Least-Once	Retry senaryosunda duplicate oluşabilir. Kabul edilebilir mi? Uygulama katmanında karar verilmelidir.
Cross-DB Bulk Writer (COPY / BCP / OCI_BATCH)	Egress Queue + native bulk API + checkpoint (ordering key opsiyonel)	At-Least-Once	Bulk write API'leri doğası gereği idempotent değildir. Checkpoint ile kaldığı yerden devam sağlanır; ancak yeniden çalışmada hedef tabloda duplicate oluşabilir. load_method: upsert veya truncate+insert ile bu risk giderilebilir.
Cross-DB Bulk Writer + load_method: upsert	Egress Queue + native bulk API + checkpoint + UPSERT hedef mantığı	Effectively Exactly-Once	Teknik olarak at-least-once; upsert (INSERT … ON CONFLICT UPDATE) sayesinde duplicate yerine güncelleme yapılır. Pratik sonuç exactly-once gibi davranır ancak sıra garantisi yoktur.
Cross-DB Bulk Writer + load_method: truncate+insert	Egress Queue + native bulk API + TRUNCATE + INSERT (her çalışmada)	Exactly-Once (İdempotent Yeniden Yüklenebilir)	Her çalışma tabloyu tamamen yenilediği için retry senaryosunda duplicate oluşmaz. Ancak partial failure durumunda hedef tablo boş kalabilir — staging senaryoları için uygundur.
Checkpoint YOK (sadece	Egress Queue + native	Best-Effort / At-	Failure sonrası

Kombinasyon	Aktif Bileşenler	Delivery Semantiği	Notlar
basit bulk transfer)	bulk API (checkpoint devre dışı)	Most-Once	kaldığı yerden devam edilemez. Kritik iş yüklerinde önerilmez.

Uygulama Rehberi: Kritik finansal veriler için önerilen kombinasyon: Queue-backed + Idempotent Sink (UPSERT) + Ordering Key + Checkpoint. Bu kombinasyon exactly-once semantiğine en yakın ve en güvenli yapıdır. Staging/DWH yüklemeleri için truncate+insert genellikle yeterlidir.

Adaptif Mikro-Batch Stratejisi

Limit Tipi	Varsayılan Değer	Açıklama
Satır Sayısı Limiti	500 satır	Paket başına maksimum satır
Byte Limiti	2 MB	Paket başına maksimum veri boyutu
Transaction Süresi Limiti	2 saniye	Hangisi önce dolarsa paket kapatılır
Hedef Yanıt Adaptasyonu	Dinamik	Hedef sistem yavaşlarsa lane başına batch küçültülür
Lock Süresi Adaptasyonu	Dinamik	Lock süresi artarsa concurrency otomatik düşürülür

Akıllı Akış Kontrolü: 500 satır VEYA 2 MB VEYA 2 saniye hangisi önce dolarsa paket kapatılır; hedef sistem yavaşlarsa batch küçültülür, lock süresi artarsa concurrency düşürülür. FFEngine'i 'sadece entegrasyon aracı' değil, akıllı akış kontrolü yapan entegrasyon platformu yapar.

Multi-Lane Pipeline Mimarisi — YAML Yapılandırması

```yaml
# Multi-Lane Pipeline Tanımı
lanes:
  - lane_id: "orders_lane_1"
    source_adapter: "oracle_aq"
    target_writer: "pg_micro_batch"
    checkpoint: true
    pool_limit: 3
    retry_policy: "exponential_backoff"
    ordering_key: "order_id"
    batch_policy:
      max_rows: 500
      max_bytes: "2MB"
      max_time: "2s"

  - lane_id: "payments_lane_1"
    source_adapter: "sql_polling"
    target_writer: "mssql_micro_batch"
    checkpoint: true
    pool_limit: 2
    retry_policy: "exponential_backoff"
```

Airflow'un Queue-Centric Mimarideki Rolü

Airflow Özelliği	Queue Mimarisindeki Kullanımı
Dynamic Task Mapping	Çalışma zamanında veri/parça sayısına göre görevler çoğaltılır; küçük paketler paralel lane'lere bölünür
Deferrable Operators	Bekleyen/polling yapan task'lar worker slot işgal etmeden triggerer üzerinde bekler; çok sayıda küçük queue akışı için kritik
Airflow Pools	Aynı anda belirli bir sisteme kaç task yüklenebileceğini sınırlar; hedef DB üzerindeki baskıyı kontrol eder
XCom Metrics	Queue depth, lag ve throughput metriklerini takip eder; sonraki turu buna göre planlar

Kritik Sorumluluk Ayrımı: Airflow queue broker değildir, orchestration yapar. Mesaj saklama, ack, redelivery, ordering ve DLQ mantığının merkezi FFEngine olmalıdır. Airflow: pipeline başlat, lane'leri üret, health/timeout/retry orchestration yap. FFEngine: checkpoint, at-least-once delivery (varsayılan), source offset, target write ack, poison message yönetimi ve idempotency. Exactly-once yalnızca Delivery Guarantee Matrix'teki koşullar sağlandığında elde edilebilir.

DBMS Bazında Queue Destekleri

DBMS	Native Queue Teknolojisi	Alternatif Yaklaşım	Kullanım Senaryosu
Oracle	AQ (Advanced Queuing) / TxEventQ	—	Producer-consumer mantığını veritabanı içine entegre biçimde sağlar
PostgreSQL	COPY TO STDOUT / FROM STDIN (Binary Queue Pipeline)	Queue Table + SKIP LOCKED / LISTEN-NOTIFY	COPY protokolü ile binary bloklar kuyruk üzerinden hem okunur hem yazılır
MSSQL	Service Broker	CDC ile polling	Veritabanı seviyesinde mesajlaşma altyapısı
Harici (Gelecek Faz)	Kafka / RabbitMQ	—	Harici broker entegrasyonu — mevcut fazda kapsam dışı; ilerideki bir Enterprise fazında değerlendirilecek

05 — Dialect Soyutlama Katmanı (Agnostic Layer)

Her veritabanı desteği için BaseDialect sınıfından türetilen sınıflar yazılır. Bu yaklaşım, çekirdek kodu veritabanı bağımsız kılar.

BaseDialect Arayüzü

```
class BaseDialect:
    def get_table_schema(self, schema: str, table: str) -> list[dict]:
        raise NotImplementedError  # Kaynaktan metadata çekme sorgusu

    def generate_bulk_insert_query(self, schema: str, table: str, columns: list)
-> str:
        raise NotImplementedError  # Hedef veritabanının en hızlı INSERT
sentaksını üretme

    def get_data_type_map(self) -> dict:
        raise NotImplementedError  # Kaynaktaki X tipinin hedefteki Y tipine
karşılığı

    def generate_ddl(self, table_meta: dict) -> str:
        raise NotImplementedError  # Kaynak metadata'dan hedef CREATE TABLE DDL
üretimi

    def get_pagination_query(self, base_sql: str, offset: int, limit: int) ->
str:
        raise NotImplementedError  # DB-specific pagination (OFFSET/FETCH,
ROWNUM, ROW_NUMBER)

class PostgresDialect(BaseDialect):
    def generate_bulk_insert_query(self, schema, table, columns):
        # Enterprise Only: COPY FROM STDIN (FORMAT BINARY)
        # Community: bu metod standart INSERT ... VALUES döndürür
        return f"COPY {schema}.{table} ({','.join(columns)}) FROM STDIN (FORMAT
BINARY)"

    def generate_bulk_extract_query(self, inner_sql: str) -> str:
        """Kaynak PG'den binary COPY TO STDOUT — Enterprise Only"""
        return f"COPY ({inner_sql}) TO STDOUT (FORMAT BINARY)"

class MSSQLDialect(BaseDialect):
    def generate_bulk_insert_query(self, schema, table, columns):
        # Enterprise Only: fast_executemany=True / BCP
        # Community: bu metod standart INSERT ... VALUES döndürür
        placeholders = ','.join(['?'] * len(columns))
        return f"INSERT INTO [{schema}].[{table}] VALUES ({placeholders})"

class OracleDialect(BaseDialect):
    def generate_bulk_insert_query(self, schema, table, columns):
        # Enterprise Only: OCI_BATCH_ERRORS
        # Community: bu metod standart executemany döndürür
        placeholders = ','.join([f':{i+1}' for i in range(len(columns))])
        return f"INSERT INTO {schema}.{table} VALUES ({placeholders})"
```

Veri Tipi Eşleme Tablosu (TypeMapper)

Oracle Tipi	MSSQL Tipi	PostgreSQL Tipi	Notlar
NUMBER(p,s)	DECIMAL(p,s)	NUMERIC(p,s)	Finansal hassasiyet için p,s korunmalı
NUMBER(16,0)	BIGINT	BIGINT	Scale=0 ise integer

Oracle Tipi	MSSQL Tipi	PostgreSQL Tipi	Notlar
			tipi
VARCHAR2(n)	NVARCHAR(n)	VARCHAR(n)	Unicode desteği kontrol edilmeli
DATE	DATETIME2	TIMESTAMP	Sürücü seviyesinde datetime objesi
CLOB	NVARCHAR(MAX)	TEXT	LOB chunking stratejisi gerekli
BLOB	VARBINARY(MAX)	BYTEA	LOB chunking stratejisi gerekli
TIMESTAMP(n)	DATETIME2(n)	TIMESTAMP(n)	Mikrosaniye hassasiyeti
CHAR(n)	CHAR(n)	CHAR(n)	Trailing space davranışı farklı

Metadata API Farklılıkları

Veritabanı	Metadata API / View	Dialect Sınıfı
Oracle	ALL_TAB_COLUMNS, ALL_TABLES	OracleDialect
PostgreSQL	information_schema.columns, pg_catalog	PostgresDialect
MSSQL	sys.columns, sys.objects, INFORMATION_SCHEMA	MSSQLDialect

06 — Modüler Yazılım Mimarisi (Bileşen Detayları)

Ana Modüller

Modül	Sorumluluk	Alt Bileşenler
Core Engine	Akış kontrolü, konfigürasyon okuma ve hata yönetimi	ETLManager, ConfigParser, ErrorHandler, ProgressTracker
Dialect Manager	Veritabanına özgü SQL üretim motoru	PostgresDialect, OracleDialect, MSSQLDialect, TypeMapper
Queue Runtime (ENTERPRISE)	Source Producer ve Target Consumer arasında çift taraflı kuyruk köprüsü kuran çekirdek modül. Community versiyonunda bu modül yoktur; yerine klasik Streamer (generator pattern) kullanılır.	IngressQueue, EgressQueue, SourceAdapter, TargetConsumer, BackpressureController, CheckpointStore, DeliveryManager
Metadata Inspector	Kaynak tablonun şema bilgisini sorgulayan dedektif katmanı	SchemaReader, DDLGenerator, IndexAnalyzer
DBSession	Veritabanı sürücüsünü sarmalayan context manager. Bağlantı açma/kapama, transaction ve cursor yönetimini standartlaştırır.	DBSession (etl_base_project/src/db/session.py)
Dynamic Filters	Binding parametrelerini çözümleyen modül. source, target, literal veya airflow_var kaynaklarından değer alarak WHERE koşullarını dinamik olarak oluşturur.	resolve_where_with_bindings(), BindingResolver
Config Validator	YAML konfigürasyonunu çalışma öncesi doğrulayan modül.	validate_config()

DBSession Context Manager

```
class DBSession:
    """Driver-agnostic DB connection context manager."""
    def __init__(self, connection_params: dict, dialect: BaseDialect):
        self.params = connection_params
        self.dialect = dialect
        self.conn = None

    def __enter__(self):
        self.conn = self.dialect.connect(self.params)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            self.conn.rollback()
        else:
            self.conn.commit()
        self.conn.close()

    def cursor(self, server_side=False):
```

```
        return self.dialect.create_cursor(self.conn, server_side)
```

Transformer TABLE_RULES Yapılandırması

Kural	Açıklama	Örnek Kullanım
strip	String kolonlardaki baştaki ve sondaki boşlukları temizler	VARCHAR tipindeki tüm kolonlara otomatik
parse_dates	Tarih string'lerini datetime nesnesine dönüştürür	Farklı tarih formatları arasında uyumluluk
to_numeric	String olarak gelen sayısal değerleri numeric'e cast eder	CSV import veya LOB field dönüşümleri
empty_to_null	Boş string ('') değerleri NULL'a çevirir	CSV/text kaynaklarda boş alan yönetimi
add_etl_date	Otomatik ETL tarihi kolonu ekler	Audit ve lineage takibi için her satıra timestamp

validate_config() — Konfigürasyon Doğrulama

Kontrol	Açıklama	Hata Durumu
source_type validasyonu	İzin verilen değerler: table, sql, csv (script ayrıca kontrol edilir)	Geçersiz source_type hatası
column_mapping_mode validasyonu	İzin verilen değerler: source, mapping_file	Geçersiz mapping mode hatası
Zorunlu alan kontrolü	task_group_id, source_schema, target_schema, target_table gibi alanlar	Missing required field hatası
mapping_file varlık kontrolü	column_mapping_mode: mapping_file ise dosyanın fiziksel varlığı kontrol edilir	File not found hatası
sql_file varlık kontrolü	source_type: sql ve sql_file tanımlı ise dosyanın fiziksel varlığı kontrol edilir	File not found hatası

Load Method Seçenekleri

load_method	Davranış	Açıklama
create_if_not_exists_or_truncate	Tablo yoksa CREATE, varsa TRUNCATE + INSERT	Varsayılan staging yükleme modu
append	Mevcut veriye ekleme yapar	Incremental yükleme için
replace	DROP + CREATE + INSERT	Tam tablo yenileme
upsert	Primary key bazlı	Delta yükleme için

load_method	Davranış	Açıklama
	INSERT/UPDATE	
delete_from_table	Koşullu DELETE + INSERT	WHERE ile hedefte belirli kayıtları silip yeniden yazar (DELETE + INSERT pattern)
drop_if_exists_and_create	DROP IF EXISTS + CREATE + INSERT	Her çalışmada tabloyu sıfırdan oluşturur
script	SQL script çalıştırma	Hedef veritabanında özel SQL script yürütür (source_type: script ile birlikte kullanılır)

07 — YAML Konfigürasyon Yapısı (Low-Code ETL)

Tüm ETL görevleri YAML dosyaları üzerinden, Python kodu yazılmadan tanımlanır. Her görev bağımsız olarak konfigüre edilebilir.

Ana Konfigürasyon Dosyası

```
# ffengine ETL Konfigürasyonu
source_db_var: ""               # Airflow Connection ID (kaynak DB)
target_db_var: ""               # Airflow Connection ID (hedef DB)

etl_tasks:
  - task_group_id: "source_schema_source_table_to_target_schema_target_table_v1"

    # ── Kaynak Tanımı ────────────────────────────────────────
    source_schema: "source_schema"
    source_table:  "source_table_name"
    source_type:   table           # table | view | sql | csv | script
    column_mapping_mode: "source"  # source | mapping_file
    sql_file: ""                   # harici .sql dosya yolu (source_type: sql
ile)

    # ── Hedef Tanımı ─────────────────────────────────────────
    target_schema: "target_schema"
    target_table:  "target_table_name"
    load_method: "create_if_not_exists_or_truncate"
    # + delete_from_table | drop_if_exists_and_create | script

    # ── Filtreleme ───────────────────────────────────────────
    where: >
      "EntryDateTime" > :dwh_oc_epr_max_entry_date_time
    order_by: ""

    # ── Dinamik Bağlamalar (Bindings) ────────────────────────
    bindings:
      dwh_oc_epr_max_entry_date_time:
        from: target              # source | target | literal | airflow_var
        sql: |
          SELECT MAX(date_trunc('day',"LastUpdatedDate"))
          FROM "Schema"."table_name"
        default: "1900-01-01 00:00:00"

    # ── Streaming Parametreleri ──────────────────────────────
    passthrough_full:   true
    passthrough_format: "binary"  # binary | text | csv
    pipe_queue_max:     8
    batch_size:         10000

    # ── Queue-Based Streaming (Enterprise / PG Binary) ──────
    extraction_method: "copy_binary"  # copy_binary | cursor | auto
    reader_workers:    3
    writer_workers:    5

    # ── Partitioning Stratejisi ──────────────────────────────
    partitioning:
      enabled: true
      column:  "MemberId"
      mode:    "percentile"  # auto | percentile | hash_mod | explicit |
distinct
      parts:   2

    # ── DAG Metadata ─────────────────────────────────────────
    tags: ["whk", "level1", "src_to_stg"]  # klasör yapısından otomatik veya
UI'den
```

Konfigürasyon Parametreleri Referansı

Parametre	Tip	Zorunlu	Varsayılan	Açıklama
task_group_id	string	✓	—	Benzersiz görev kimliği, dosya adlandırma ile uyumlu
source_schema	string	✓	—	Kaynak DB şema adı
source_table	string	✓	—	Kaynak tablo adı
source_type	enum	—	table	table / view / sql / csv / script
column_mapping_mode	enum	—	source	source (kaynak kolon aynen) / mapping_file (YAML mapping dosyası)
sql_file	string	—	—	Harici .sql dosya yolu (source_type: sql ile). ETL Studio da SQL'i ayrı dosyaya yazar.
tags	list	—	[]	DAG tag'leri. Klasör yapısından otomatik türetilir veya UI'den eklenir.
target_schema	string	✓	—	Hedef DB şema adı
target_table	string	✓	—	Hedef tablo adı
load_method	enum	—	truncate	create_if_not_exists_or_truncate / append / replace / upsert / delete_from_table / drop_if_exists_and_create / script
where	string	—	—	SQL WHERE koşulu, binding parametreleri ile
bindings	map	—	—	Dinamik parametre eşlemeleri (source / target / literal / airflow_var)
passthrough_format	enum	—	binary	binary / text / csv
pipe_queue_max	int	—	8	Backpressure buffer boyutu
batch_size	int	—	10000	fetchmany chunk boyutu (satır)
extraction_method	enum	—	auto	copy_binary (Enterprise Only — PG COPY TO STDOUT) / cursor (Community + Enterprise — fetchmany) / auto
reader_workers	int	—	3	Paralel okuyucu worker sayısı
writer_workers	int	—	5	Paralel yazıcı worker sayısı
partitioning.mode	enum	—	auto	auto / percentile / hash_mod / explicit / distinct
partitioning.parts	int	—	2	Partition parça sayısı

08 — Mapping Dosya Yapısı

Mapping yapısı, kaynak-hedef kolon eşleşmesini dosya sistemi hiyerarşisi üzerinden standartlaştırır. Her görev için bağımsız bir mapping dosyası tutulur.

Dizin Yapısı

```
mappings/
  └── {source_schema}/
        └── {source_table}_to_{target_schema}_{target_table}_v{version}.yaml

# Örnek:
mappings/
  └── ocn_iss/
        ├──
crd_account_balance_used_hist_to_dwh_stg_crd_account_balance_used_hist_v1.yaml
        └── txn_issuer_security_to_dwh_stg_txn_issuer_security_v1.yaml
```

Mapping Dosyası — Tablo Bazlı (column mapping)

```yaml
version: '1.0'
mapping_type: column
generated_at: '2025-10-07T10:14:49Z'
source:
  schema: ocn_iss
  table:  crd_account_balance_used_hist
columns:
  - source: guid
    target: guid
    type:   NUMERIC(16, 0)
  - source: last_updated
    target: last_updated
    type:   NUMERIC(16, 0)
  - source: txn_guid
    target: txn_guid
    type:   NUMERIC(16, 0)
  - source: txn_def_guid
    target: txn_def_guid
    type:   NUMERIC(16, 0)
  - source: account_no
    target: account_no
    type:   VARCHAR(10)
```

Mapping Dosyası — SQL Bazlı (custom SQL sorgusu)

```yaml
version: '1.0'
mapping_type: column
generated_at: '2025-10-08T12:10:10Z'
source:
  schema: ocn_iss
  table:  txn_issuer_security
  sql: |
    SELECT
      "ocn_iss"."txn_issuer_security"."txn_guid",
      "ocn_iss"."txn_issuer_security"."cavv_ind",
      "ocn_iss"."txn_issuer_security"."cavv_data",
      "ocn_iss"."txn_issuer_security"."security_level_indicator"
    FROM "ocn_iss"."txn_issuer_security"
    WHERE EXISTS (
      SELECT 1 FROM ocn_iss.txn_issuer_onl tio
      WHERE txn_issuer_security.txn_guid = tio.guid
    )
```

```
columns:
  - source: txn_guid
    target: txn_guid
    type:   NUMERIC(16, 0)
  - source: cavv_ind
    target: cavv_ind
    type:   NUMERIC(3, 0)
  - source: cavv_data
    target: cavv_data
    type:   VARCHAR(48)
  - source: security_level_indicator
    target: security_level_indicator
    type:   NUMERIC(3, 0)
```

Mapping Dosya Parametreleri

Alan	Tip	Açıklama
version	string	Şema versiyonu (ör: '1.0')
mapping_type	enum	column — kolon bazlı eşleme
generated_at	ISO 8601	Otomatik üretim timestamp'i
source.schema	string	Kaynak şema adı
source.table	string	Kaynak tablo adı
source.sql	string	Custom SQL sorgusu (opsiyonel; tablo yerine)
columns[].source	string	Kaynak kolon adı
columns[].target	string	Hedef kolon adı
columns[].type	string	Hedef veri tipi (TypeMapper ile uyumlu)

Mapping Üretimi: mapping_generator.py CLI aracı ile kaynak tablo veya sorgudan otomatik YAML mapping dosyası üretilir. Kaynak DB'ye bağlanarak kolon adları, veri tipleri ve nullable bilgilerini çeker. ETL Studio UI üzerinden de mapping oluşturma desteklenir. Manuel düzenleme için sürümleme (v1, v2...) ve Git uyumlu isimlendirme önerilir.

09 — Apache Airflow Entegrasyonu

ETL Studio, Apache Airflow arayüzüne Admin > ETL Studio menüsü altında Airflow plugin olarak entegre edilen özel bir eklentidir. Kullanıcılar YAML konfigürasyonunu ve DAG dosyasını bu ekran üzerinden görsel olarak oluşturabilir, mevcut DAG'leri güncelleyebilir ve çalışan DAG'lerin timeline görünümünü takip edebilir.

ETL Studio & Airflow Bileşenleri

Bileşen	Açıklama	Airflow Versiyonu
FFEngineOperator	Custom Airflow operatörü. YAML konfigürasyonunu okuyarak ffengine motorunu tetikler.	3.1.6+
Auto-DAG Generator	YAML konfigürasyonlarından otomatik Airflow DAG üretir.	3.1.6+
Dynamic Task Mapping	Veri hacmine göre otomatik worker artırımı (Horizontal Scaling).	2.3+
XCom Integration	rows_transferred, duration_seconds, rows_per_second metrikleri.	Tüm versiyonlar
ETL Studio UI	Admin > ETL Studio menüsü — şema/tablo keşfi (50 tablo limiti, typeahead arama), DAG oluşturma/güncelleme, SQL dosya yönetimi ve tag ekleme.	3.1.6+

ETL Studio UI Özellikleri

Özellik	Açıklama
Şema/Tablo Keşfi	Kaynak veritabanından şema ve tablo listesi çekilir. 50 tablo limiti ile sayfalama, arama filtresi harf yazdıkça (typeahead) çalışır.
DAG Oluşturma	UI üzerinden YAML konfigürasyonu ve DAG dosyası görsel olarak oluşturulur. SQL sorgusu ayrı .sql dosyasına yazılır.
DAG Güncelleme	Daha önce UI ile oluşturulan DAG'ler sonradan güncellenebilir.
Tag Yönetimi	Klasör yapısına göre otomatik tag eklenir. UI üzerinde de tag ekleme/düzenleme alanı vardır.
Timeline Görünümü	Çalışan DAG'ler Airflow üzerinde ayrı bir bölümde timeline olarak görsel sunulur.

FFEngineOperator Şablonu

```
from airflow.models import BaseOperator
from airflow.utils.decorators import apply_defaults

class FFEngineOperator(BaseOperator):
    """
    ffengine Custom Airflow Operator.
    Community ve Enterprise motorlarını otomatik tespit eder (automatic
fallback).
```

```
    """
    @apply_defaults
    def __init__(
        self,
        config_path: str,       # YAML konfigürasyon dosyası yolu
        source_conn_id: str,    # Airflow Connection ID (kaynak)
        target_conn_id: str,    # Airflow Connection ID (hedef)
        task_group_id: str,     # Çalıştırılacak task group
        engine: str = "auto",   # "community" | "enterprise" | "auto"
        **kwargs
    ):
        super().__init__(**kwargs)
        self.config_path    = config_path
        self.source_conn_id = source_conn_id
        self.target_conn_id = target_conn_id
        self.task_group_id  = task_group_id
        self.engine         = engine

    def execute(self, context):
        engine = _detect_engine(self.engine)  # automatic fallback
        result = engine.run(self.config_path, self.task_group_id)

        self.xcom_push(context, key='rows_transferred', value=result.rows)
        self.xcom_push(context, key='duration_seconds', value=result.duration)
        self.xcom_push(context, key='rows_per_second',  value=result.throughput)
        return result
```

9B — DAG Execution Pattern (3 Fazlı)

Her etl_task bir Airflow TaskGroup içinde 3 fazlı bir DAG pattern ile çalışır. Bu pattern, partition'lı paralel aktarımın temelini oluşturur.

3 Fazlı Execution Sırası

Faz	Task	Sorumluluk	Çıktı
Faz 1 — Plan	plan_partitions	Binding parametrelerini çözer (resolve_where_with_bindings), partition stratejisini belirler (AutoNumeric / Percentile / HashMod / Distinct / Explicit / FullScan), partition spec listesi üretir.	XCom → partition_specs listesi (her biri min/max veya WHERE koşulu içerir)
Faz 2 — Prepare	prepare_target	Hedef tabloyu hazırlar: load_method'a göre CREATE TABLE / TRUNCATE / DROP+CREATE / DELETE gibi DDL işlemlerini yürütür.	Hedef tablo veri almaya hazır durumda
Faz 3 — Run	run_partition.expand()	Her partition spec için ETLManager.run_etl_task() paralel olarak çalıştırılır. Airflow Dynamic Task Mapping ile her partition ayrı bir worker'da çalışır.	XCom → rows_transferred, duration, throughput (her partition için)

3 Fazlı DAG Pattern — Kod Örneği

```
from airflow.decorators import dag, task, task_group
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

for etl_task in config["etl_tasks"]:
    with TaskGroup(group_id=etl_task["task_group_id"]) as tg:

        @task
        def plan_partitions(task_cfg):
            # 1. Binding'leri çöz (source/target/literal/airflow_var)
            resolved = resolve_where_with_bindings(task_cfg)
            # 2. Partition stratejisi belirle ve spec listesi üret
            specs = partitioner.generate(resolved)
            return specs  # XCom → [{"where": "id BETWEEN 0 AND 50000"}, ...]

        @task
        def prepare_target(task_cfg):
            # Hedef tabloyu hazırla (CREATE/TRUNCATE/DROP/DELETE)
            target_writer.prepare(task_cfg["load_method"])

        @task
        def run_partition(partition_spec, task_cfg):
            # ETLManager her partition'ı bağımsız olarak çalıştırır
            ETLManager.run_etl_task(task_cfg, partition_spec)

        specs = plan_partitions(etl_task)
        prep  = prepare_target(etl_task)
        # Dynamic Task Mapping — her spec paralel worker'da çalışır
        specs >> prep >> run_partition.expand(partition_spec=specs)
```

Binding Çözümleme Mekanizması (dynamic_filters)

from Değeri	Kaynak	Açıklama
source	Kaynak veritabanı	Kaynak DB üzerinde binding SQL'i çalıştırılarak değer elde edilir
target	Hedef veritabanı	Hedef DB üzerinde binding SQL'i çalıştırılarak değer elde edilir (ör: MAX tarih)
literal	Sabit değer	YAML'da doğrudan tanımlanan statik değer kullanılır
airflow_var	Airflow Variable	Airflow Variables tablosundan runtime'da değer çekilir

TriggerDagRunOperator — Level1 → Level2 Kademeli Tetikleme

Çok katmanlı ETL pipeline'larında Level1 DAG'ları (src→stg) tamamlandıktan sonra Level2 DAG'ları (stg→final) otomatik olarak tetiklenir.

```
# Level1 DAG'ı tamamlandığında Level2'yi tetikle
trigger_level2 = TriggerDagRunOperator(
    task_id="trigger_level2_stg_to_whk",
    trigger_dag_id="whk_level2_stg_to_whk",
    wait_for_completion=True,  # Level2 bitene kadar bekle
    poke_interval=30,          # 30 saniyede bir kontrol
)
# DAG akışı: tüm Level1 task'ları >> trigger_level2
all_level1_tasks >> trigger_level2
```

9C — Proje Dizin Yapısı & Konfigürasyon Hiyerarşisi

FFEngine projeleri standart bir dizin yapısı ile organize edilir. Her proje, domain ve seviye (level) bazında gruplandırılmış ETL görevlerini içerir.

```
projects/webhook/
├── whk/
│   ├── level1/
│   │   ├── src_to_stg/            # Kaynak → Staging (Level 1)
│   │   │   ├── config.yaml        # Ana YAML konfigürasyonu
│   │   │   ├── mappings/          # Tablo bazlı mapping dosyaları
│   │   │   └── sql/               # Harici SQL dosyaları
│   │   └── src_to_ods/            # Kaynak → ODS (Level 1)
│   │       ├── config.yaml
│   │       └── mappings/
│   └── level2/
│       └── stg_to_whk/            # Staging → Final (Level 2)
│           ├── config.yaml
│           └── sql/               # DELETE + INSERT script'leri
└── oc_epr/
    └── level1-2/                  # İkinci domain
        ├── config.yaml
        └── mappings/

# Klasör yapısı → DAG tag'leri otomatik türetilir:
# whk/level1/src_to_stg → tags: ["whk", "level1", "src_to_stg"]
```

Dizin Hiyerarşisi Tablosu

Dizin Seviyesi	Anlam	Örnek
projects/{proje_adı}/	Proje kök dizini	projects/webhook/
{domain}/	İş domain'i (kaynak sistem veya konu alanı)	whk/, oc_epr/
level{N}/	ETL seviyesi (1=kaynak→staging, 2=staging→final)	level1/, level2/
{akış_yönü}/	Kaynak → Hedef yön tanımı	src_to_stg/, stg_to_whk/
config.yaml	ETL görev konfigürasyonu	source_db_var, etl_tasks[]
mappings/	Tablo bazlı mapping YAML dosyaları	mapping_generator.py çıktısı
sql/	Harici SQL dosyaları (sql_file referansı)	DELETE+INSERT pattern script'leri

10 — Teknoloji Stack

Kategori	Teknoloji	Versiyon	Kullanım Amacı	Kapsam
Orkestrasyon	Apache Airflow	3.1.6	DAG yönetimi, görev planlama, izleme	Her iki versiyon
Konteyner	Docker	apache/ airflow:3.1.6-python3.12	Airflow + ffengine paketi dağıtımı	Her iki versiyon
İşletim Sistemi	Linux (Ubuntu)	—	8 CPU, 32GB RAM referans ortam	Her iki versiyon
Dil (Community)	Python	3.12	Ana geliştirme dili	Community
Dil (Enterprise)	C / C++	—	Zero-Copy Streaming çekirdeği	Enterprise
PostgreSQL Sürücüsü	psycopg3	3.x	Standart DBAPI bağlantısı (Community). Native binary COPY TO/FROM STDIN — Enterprise Only.	Her iki versiyon (farklı özellik seti)
MSSQL Sürücüsü	pyodbc + msodbcsql18	—	UnixODBC üzerinden MSSQL bağlantısı	Community
Oracle Sürücüsü	python-oracledb	—	Thin mode — Oracle Client gerektirmez	Community
PostgreSQL Native	libpq	—	C seviyesinde PostgreSQL bağlantısı	Enterprise
Oracle Native	OCI (Oracle C Interface)	—	C seviyesinde Oracle bağlantısı	Enterprise
MSSQL Native	SQL Server Native Client	—	C seviyesinde MSSQL bağlantısı	Enterprise
Konfigürasyon	YAML	—	ETL pipeline tanımlama, versiyonlama	Her iki versiyon
Güvenlik	RSA Lisans Modülü	—	Enterprise lisans doğrulama	Enterprise
Bellek Testi	Valgrind	—	C extension bellek sızıntısı testi	Enterprise
Loglama	Airflow DAG Logs (JSON)	—	ELK Stack / Splunk entegrasyonu için	Her iki versiyon

11 — Performans Hedefleri (Technical KPIs)

<500MB Sabit RAM (veri hacminden bağımsız)	**100K+** Satır/saniye throughput	**<3 dk** 10M+ satır bulk aktarım	**ms** Kaynak → hedef latency
<256MB Enterprise — 10M satır RAM	**<1 hft** Yeni connector ekleme süresi	**%30–40** Binary streaming CPU tasarrufu (Enterprise)	**5–10x** Enterprise / Community hız oranı

KPI	Community Hedef	Enterprise Hedef	Ölçüm Yöntemi
RAM Kullanımı (10M satır)	< 500MB	< 256MB	Airflow worker memory monitor
Throughput (batch / direkt aktarım, queue overhead olmadan)	100,000 satır/sn	500,000+ satır/sn	ProgressTracker (rows/sec)
Throughput (tam queue semantiği)	50–70K satır/sn	250–400K satır/sn	ProgressTracker (checkpoint+ack/nack dahil)
Bulk aktarım süresi	< 3 dakika	< 1 dakika	Airflow task duration
End-to-end latency	Milisaniyeler (chunk bazlı)	Milisaniyeler (chunk bazlı)	XCom timestamp delta
Yeni Dialect ekleme	< 1 hafta	< 1 hafta	Sprint velocity
Pipeline kurulum süresi	Saatler → Dakikalar	Saatler → Dakikalar	Kullanıcı testi

KPI Benchmark Bağlamı: Throughput ve latency hedefleri doğrudan streaming senaryosu (queue overhead olmadan, binary pipe) için geçerlidir. Enterprise'ın tam queue semantiği aktifken (checkpoint + ack/nack + delivery policy + multi-lane) throughput doğal olarak düşer. 'Milisaniyeler end-to-end latency' ifadesi chunk bazlı transfer süresini tanımlar; Airflow orchestration + 3 fazlı DAG + checkpoint dahil pipeline end-to-end süresi senaryo bağımlıdır.

12 — Hata Yönetimi & Gözlemlenebilirlik

Özellik	Davranış	Uygulama
Seviye Bazlı Loglama	Log Level 1=INFO, 2=WARNING, 3=ERROR	Airflow DAG log altyapısı, JSON formatında
Hata Aşaması İzleme	extract / transform / load etiketi ile hata loglama	ProgressTracker modülü
Partial Success Handling (Community)	100K satırdan 1 hatalı → hatalı satır loglanır, işlem devam eder. Gelişmiş DLQ yönlendirme Enterprise'a aittir.	Hatalı satır Airflow log'a yazılır. DLQ (Dead Letter Queue) yönlendirme → Enterprise Only
Real-time Metrics	rows/sec ve kalan tahmini süre Airflow log'a anlık basılır	ProgressTracker modülü
Rollback (Community)	Hata anında chunk bazlı transaction rollback. PK/Offset bazlı tam Resume: Enterprise Only.	Community: transaction rollback | Enterprise: CheckpointStore, OffsetTracking
Checkpoint & Resume (Enterprise Only)	Source offset ve target write ack takibi ile kaldığı yerden devam (resume) yeteneği. Tam Checkpoint & Resume yalnızca Enterprise Queue Runtime içinde yer alır.	Enterprise Only — CheckpointStore, OffsetTracking, idempotent-write
DLQ — Dead Letter Queue (Enterprise Only)	Hatalı paketler için exponential backoff ile yeniden deneme; maksimum deneme sonrası Dead Letter Queue'ya yönlendirme. Yalnızca Enterprise Queue Runtime içinde yer alır.	Enterprise Only — Retry & DLQ Policy, poison-message yönetimi
XCom Metrikleri	rows_transferred, duration_seconds, rows_per_second	FFEngineOperator.execute() → xcom_push()
Structured Logging	JSON formatında log (ELK / Splunk entegrasyonu için)	summary_report üretimi (her aktarım sonunda)
Dry-Run Modu	Veri aktarmadan DDL üretimi ve bağlantı testi	--dry-run flag / config parametresi
Email Notification	Hata durumunda Türkçe HTML formatında email bildirimi — DAG ID, Task ID, süre, hostname	send_email_custom.py — on_failure_callback

Valgrind Zorunluluğu (Enterprise): C tarafındaki bellek sızıntıları tüm Airflow worker'ını çökertebilir. Enterprise versiyonunda Valgrind ile stres testleri zorunludur. Her C-extension release öncesi Valgrind raporu CI/CD pipeline'ında çalıştırılmalıdır.

12B — Geliştirme Araçları & Yardımcı Modüller

VS Code Remote Debugging (debugpy)

Geliştirme ortamında wait_for_debug() fonksiyonu ile VS Code üzerinden Airflow worker'a uzaktan debug bağlantısı yapılabilir. debugpy port 5678 üzerinden dinler ve bağlantı gelene kadar bekler.

```
import debugpy

def wait_for_debug(port=5678):
    """VS Code debugger bağlantısı bekler."""
    debugpy.listen(("0.0.0.0", port))
    print(f"Waiting for debugger on port {port}...")
    debugpy.wait_for_client()  # VS Code bağlanana kadar bekle

# DAG'da kullanım:
# wait_for_debug()  # geliştirme sırasında aktif edilir
```

mapping_generator CLI Aracı

Özellik	Açıklama
Otomatik Şema Keşfi	Kaynak tablonun kolon adları, veri tipleri ve nullable bilgilerini çeker
Custom SQL Desteği	Tablo adı yerine SQL sorgusu verilerek sorgu sonucunun mapping'i üretilir
Sürümleme	Dosya adına otomatik _v1, _v2 eklenir — Git uyumlu isimlendirme
Hedef Dizin	Proje dizin yapısına uygun mappings/ klasörüne çıktı üretir

Email Notification Plugin (send_email_custom.py)

send_email_custom.py modülü, Airflow on_failure_callback olarak çalışır ve hata durumunda Türkçe HTML formatında email gönderir.

Email İçeriği	Açıklama
DAG ID	Hata veren DAG'ın kimliği
Task ID	Hata veren task'ın kimliği
Süre	Task'ın çalışma süresi
Hostname	Worker'ın çalıştığı sunucu bilgisi
Hata Detayı	Exception stacktrace ve hata mesajı

13 — İleri Düzey Teknik Detaylar

LOB (Large Objects) Yönetimi

Oracle Tipi	PostgreSQL Tipi	Strateji
CLOB	TEXT	Ayrı chunking stratejisi — belirli boyut limitine kadar destek veya LOB chunking
BLOB	BYTEA	Ayrı chunking stratejisi — belirli boyut limitine kadar destek
NCLOB	TEXT	Unicode encoding dikkatli yönetilmeli

Finansal Veri Hassasiyeti: Oracle 38 basamak hassasiyet desteğine karşın PostgreSQL NUMERIC sınırları farklıdır. TypeMapper içinde finansal DECIMAL/NUMERIC tipler için hassasiyet (precision) ve ölçek (scale) kayıpsız aktarım garantisi tanımlanmalıdır.

Paralelizasyon Stratejisi

Strateji	Açıklama	Uygulama
Vertical Scaling	Tek ffengine instance içinde multiprocessing ile farklı partition'ları paralel çekme	Python multiprocessing, partition bazlı task split
Horizontal Scaling	Airflow Dynamic Task Mapping ile tabloyu parçalara bölüp her parçayı farklı Airflow Worker'a dağıtma	Airflow 2.3+ Dynamic Task Mapping API
SIMD (Enterprise)	İşlemci seviyesinde SIMD talimat setleri ile veri dönüşümü paralelleştirme	C/C++ SIMD intrinsics (SSE4.2 / AVX2)

14 — Geliştirme Fazları (Sprint Planı)

Community GA — Tek Fazda Lansman (Q1 2026)

Community GA Hedef: Airflow-Native Low-Code ETL Platform — kullanıcılar ekran üzerinden kod yazmadan DAG oluşturup aktarımlarını gerçekleştirir.

Başlık	Teslim Edilecek Çıktı
Engine	Python Engine — fetchmany ile okuma, executemany ile yazma (standart DBAPI). Tüm desteklenen veritabanları arasında çift yönlü aktarım.
Dialect	PostgreSQL, MSSQL, Oracle dialect sınıfları + Airflow'un desteklediği tüm connection tipleri ile çalışabilir agnostic yapı. TypeMapper, column_mapping_mode, validate_config().
Airflow	FFEngineOperator, Auto-DAG Generator, 3 Fazlı DAG Execution Pattern (plan → prepare → run), Dynamic Task Mapping, TriggerDagRunOperator, Airflow Pools.
ETL Studio UI	Admin > ETL Studio — şema/tablo keşfi, DAG oluşturma/güncelleme, tag yönetimi, timeline görünümü, SQL dosya yönetimi.
Konfigürasyon	YAML tabanlı pipeline tanımlama, partition desteği, binding çözümleme (source/target/literal/airflow_var), mapping_generator CLI.
Operasyonel	Email notification (on_failure_callback), Dry-Run modu, Structured Logging (JSON), chunk bazlı transaction rollback. Not: Gelişmiş Checkpoint & Resume ve DLQ → Enterprise Only.

Enterprise GA — Tek Fazda Lansman (Q2 2026)

Enterprise GA Hedef: Queue-Centric High-Performance Engine — Community üzerine Engine Swap ile C Engine devreye alınır. UI, Dialect, DB bağlantıları ve Airflow entegrasyonu değişmez.

Başlık	Teslim Edilecek Çıktı
Engine Swap	Community'deki Python Engine yerine C Engine devreye alınır.
Queue Runtime	Ingress Queue + Egress Queue çift taraflı kuyruk altyapısı (okuma VE yazma tarafı); Source Producer + Target Consumer; backpressure ve checkpoint; FFEnvelope, Ack/Nack contract, Delivery Policy.
DB'ye Özel Hızlı Aktarım	PostgreSQL binary COPY, MSSQL BCP, Oracle OCI_BATCH — native bulk API'ler ile yüksek hızlı yazma.
Gelişmiş Akış Kontrolü	Adaptif Mikro-Batch stratejisi (satır + byte + süre + feedback); Multi-Lane Pipeline; Parametrik N Reader / M Writer.
C-Core Performans	Zero-Copy Streaming; Native sürücüler (libpq, OCI, SQL Server Native Client); SIMD optimizasyonu.
Gelecek Genişleme	Snowflake / BigQuery C-connectors; Native queue adapters (Oracle AQ, MSSQL Service Broker); Hybrid Cloud Streaming paketleri.

15 — Mimari Karar Kaydı (ADR)

ADR ID	Karar	Gerekçe
ADR-001	Neden Python (Community)?	Ekosistemin genişliği, Apache Airflow uyumu ve psycopg3/python-oracledb gibi kütüphanelerin yüksek performansı nedeniyle. Standart DBAPI 2.0 ile geniş DB desteği hızlıca sağlanabilir.
ADR-002	Neden C/C++ (Enterprise)?	Python nesnesi oluşturma maliyetini sıfıra indiren Zero-Copy Streaming için; SIMD talimat setleri ile işlemci seviyesinde paralelizasyon için ve native kütüphaneler (OCI, libpq) üzerinden maksimum throughput için.
ADR-003	Neden YAML Konfigürasyonu?	İnsan okunabilirliği, Git versiyon kontrolü dostu olması ve teknik olmayan ekiplerin (Data Analyst) de konfigürasyon yapabilmesi için.
ADR-004	Neden Binary Streaming (Enterprise)?	Standart string tabanlı aktarımlardaki escaping ve parsing yükünü minimize etmek için. COPY, BCP ve OCI_BATCH CPU kullanımını %30–40 azaltır. Community'de bu yük mevcuttur; Enterprise'da C Engine ve native bulk API ile tamamen ortadan kalkar.
ADR-005	Neden Server-Side Cursor (fetchmany)?	Büyük tabloların tamamının belleğe alınmasını (OOM hatası) önlemek için. fetchmany(10000) ile sabit bellek kullanımı garantilenir.
ADR-006	Neden Apache Airflow (Orkestrasyon)?	Kurumsal ortamlarda yaygın kullanım, Dynamic Task Mapping ile horizontal scaling, zengin monitoring ekranları ve XCom ile metrik paylaşımı sağlaması.
ADR-007	Neden Queue-Centric Mimari?	Hedef DBMS'e doğrudan büyük batch yazma yerine queue tabanlı kontrollü yazma tercih edilmiştir. Bu yaklaşım, uzun transaction'ları, lock baskısını, WAL/redo log yükünü ve deadlock riskini minimize eder.
ADR-008	Neden Adaptif Mikro-Batch?	Sabit batch boyutu yerine satır sayısı, byte limiti ve transaction süresi limitinin birlikte değerlendirildiği adaptif strateji benimsenmiştir. Bu FFEngine'i statik ETL'den akıllı akış kontrol platformuna dönüştürür.
ADR-009	Neden Multi-Lane Pipeline?	Her tablo/akış için bağımsız lane tanımlanarak checkpoint, pool limiti, retry ve ordering key izolasyonu sağlanır. Airflow Dynamic Task Mapping ile lane'ler çalışma zamanında çoğaltılır.
ADR-010	Neden COPY TO STDOUT (FORMAT BINARY) — Enterprise Only?	PostgreSQL kaynak olduğunda (Enterprise), fetchmany() yerine COPY (subquery) TO STDOUT (FORMAT BINARY) kullanılarak veri Python nesnesine dönüştürülmeden ham binary bloklar halinde okunur. PG→PG senaryosunda uçtan uca binary pipe ile sıfır-dönüşüm sağlanır. Community'de PG→PG aktarımı da standart fetchmany + executemany ile gerçekleştirilir.
ADR-011	Neden Asimetrik Parametrik Worker Yapısı?	Kaynak ve hedef sistemlerin I/O kapasiteleri genellikle farklıdır. reader_workers ve writer_workers parametrelerinin bağımsız olması sayesinde her iki

ADR ID	Karar	Gerekçe
		tarafın kapasitesi en verimli şekilde kullanılır.
ADR-012	Neden 3 Fazlı DAG Execution Pattern?	plan_partitions → prepare_target → run_partition(expand) yapısı, partition planlama ve hedef hazırlama işlemlerini veri aktarımından ayırır. Binding çözümleme plan fazında yapılarak worker'ların bağımsızlığı sağlanır.
ADR-013	Neden column_mapping_mode Ayrımı?	Basit aktarımlarda (source modunda) mapping dosyası yazma zorunluluğu kaldırılır. Karmaşık dönüşümlerde (mapping_file modunda) YAML mapping dosyası uygulanır.
ADR-014	Neden TriggerDagRunOperator ile Kademeli Tetikleme?	Çok katmanlı ETL pipeline'larında Level2'nin Level1 bitmeden başlamaması kritik tutarlılık gereksinimi. TriggerDagRunOperator(wait_for_completion=True) ile bu sıralama deklaratif olarak sağlanır.

16 — Proje Kapsamı & Sınırlar

Kapsam Dahili — Community (Faz 1)

Community Kapsam Dahili Konular
PostgreSQL ↔ MSSQL ↔ Oracle (çift yönlü) + YAML tabanlı ETL konfigürasyonu
Server-Side Cursor (fetchmany + executemany) + Auto-DDL Generator + TypeMapper
Partitioning (auto/percentile/hash_mod/explicit/distinct)
Apache Airflow Operatörü (FFEngineOperator) + Auto-DAG Generator
ETL Studio UI — Admin > ETL Studio menüsü
Dynamic Task Mapping (horizontal scaling) + Airflow Pools
3 Fazlı DAG Execution Pattern (plan/prepare/run) + TriggerDagRunOperator
TaskGroup ile görev gruplandırma + In-flight veri tipi doğruluğu kontrolü
Chunk bazlı transaction rollback (hata anında) — Checkpoint & Resume ve DLQ: Enterprise Only
column_mapping_mode (source / mapping_file) + validate_config() + DBSession context manager
mapping_generator CLI aracı + Transformer TABLE_RULES
Email Notification Plugin + VS Code debugpy Remote Debugging
ODBC / JDBC bağlantı seçenekleri

Kapsam Dahili — Enterprise (Faz 2, Community'ye ek olarak)

Enterprise Kapsam Dahili Konular
Queue-Centric Mimari (Source Ingress / Target Egress)
Binary Streaming: COPY TO STDOUT / FROM STDIN (PG), BCP (MSSQL), OCI_BATCH (Oracle) — Enterprise Only
Multi-Lane Pipeline (paralel kuyruk akışları)
Adaptif Mikro-Batch Stratejisi + FFEnvelope / Ack-Nack / Delivery Policy
Checkpoint & Resume (PK/Offset bazlı, Source Offset + Target Write Ack takibi)
DLQ — Dead Letter Queue: exponential backoff retry + poison-message yönlendirme
Asimetrik Parametrik Worker (N Reader / M Writer)
COPY TO STDOUT (FORMAT BINARY) — PG Binary Extraction (Enterprise Only)
Oracle AQ / TxEventQ Entegrasyonu + PostgreSQL Queue Table (SKIP LOCKED)
Zero-Copy Streaming (C çekirdeği) + Native Sürücüler (libpq, OCI, SQL Server Native Client) + SIMD Optimizasyonu

Kapsam Dışı (Out of Scope)

Kapsam Dışı Konular	Gerekçe
Change Data Capture (CDC / WAL / Redo Log)	ffengine High-Speed Batch odaklıdır; CDC karmaşıklık artırır
İş kuralı bazlı veri kalitesi (null kontrol, regex)	Akış anında tip doğruluğu dışında kapsam genişler
Veri maskeleme / anonimleştirme (KVKK/GDPR)	Hassas kolon maskeleme ayrı bir süreç gerektirir
Streaming araç entegrasyonu (Kafka, RabbitMQ vb.)	Mevcut fazda kapsam dışı. Harici broker entegrasyonu Enterprise'ın gelecek bir fazında değerlendirilecek — şu an 'kalıcı kapsam dışı' değil, 'ertelenmiş' olarak sınıflandırılmıştır.
NoSQL veritabanları (MongoDB, Cassandra)	İlk sürümde RDBMS odaklı
Veri kataloğu / data governance	Ayrı ürün kapsamı

Beklenen Gelecek Kapsam (2027+): Snowflake ve BigQuery C-Konnektörleri; On-premise → Cloud Hybrid Streaming; MySQL Dialect; Kafka / RabbitMQ harici broker entegrasyonu (Enterprise gelecek faz).

17 — Gereksinimler

Fonksiyonel Gereksinimler (FR) — C=Community, E=Enterprise

ID	Kapsam	Gereksinim	Öncelik
FR01	C	YAML tabanlı konfigürasyon ile ETL job oluşturma ve çalıştırma	P0 — Zorunlu
FR02	C	PostgreSQL, MSSQL, Oracle arasında çift yönlü veri aktarımı	P0 — Zorunlu
FR03	C	Server-side cursor ile sabit bellek kullanımlı veri çekme	P0 — Zorunlu
FR04	C	Kaynak metadata'dan otomatik DDL (CREATE TABLE) üretimi	P0 — Zorunlu
FR05	E	Binary streaming ile PostgreSQL COPY FROM STDIN aktarımı	P0 — Zorunlu
FR06	C	Apache Airflow FFEngineOperator entegrasyonu	P0 — Zorunlu
FR07	C	Partitioning (percentile/hash_mod/auto/explicit/distinct) desteği	P1 — Yüksek
FR08	C	Binding parametreleri ile dinamik WHERE koşulu	P1 — Yüksek
FR09	C	Backpressure yönetimi (hedef hızına göre kaynak throttling)	P1 — Yüksek
FR10	C	Auto-DAG Generator (YAML → Airflow DAG otomasyonu)	P1 — Yüksek
FR11	C	ETL Studio UI (Admin > ETL Studio embedded görsel konfigürasyon)	P1 — Yüksek
FR12	E	Checkpoint & Resume (PK/Offset bazlı kaldığı yerden devam) — Queue Runtime içinde Source Offset + Target Write Ack takibi. Enterprise Only.	P0 — Zorunlu
FR13	E	Dead Letter Queue — hatalı paketler için exponential backoff retry ve maksimum deneme sonrası DLQ yönlendirme. Enterprise Only.	P0 — Zorunlu
FR14	C	Dry-Run modu (DDL üretimi + bağlantı testi, veri aktarımı olmadan)	P2 — Orta
FR15	C	XCom üzerinden metrik aktarımı (rows, duration, throughput)	P1 — Yüksek
FR16	E	Queue-Centric hedef yazma: Target Ingress Queue ile kontrollü mikro-batch commit	P0 — Zorunlu
FR17	E	Multi-Lane Pipeline: Bağımsız lane'ler ile paralel kuyruk akışları çalıştırma	P1 — Yüksek
FR18	E	Adaptif Batch Policy: Satır sayısı, byte limiti ve süre bazlı dinamik paket boyutlandırma	P1 — Yüksek
FR19	E	FFEnvelope standardı: Ack/Nack contract, at-least-once delivery (varsayılan), exactly-once (koşullu — Delivery Guarantee Matrix'e göre idempotent sink + checkpoint + ordering key kombinasyonu gerektirir)	P1 — Yüksek
FR20	E	Oracle AQ / TxEventQ source ve sink adapter entegrasyonu	P2 — Orta

ID	Kapsam	Gereksinim	Öncelik
FR21	E	PostgreSQL Queue Table (SKIP LOCKED) sink adapter	P2 — Orta
FR22	E	COPY TO STDOUT (FORMAT BINARY) ile PostgreSQL'den kuyruk-uyumlu binary extraction	P0 — Zorunlu
FR23	E	Asimetrik parametrik worker yapısı: reader_workers ve writer_workers bağımsız ayarlanabilir	P0 — Zorunlu
FR24	C	3 Fazlı DAG Execution Pattern: plan_partitions → prepare_target → run_partition (Dynamic Task Mapping)	P0 — Zorunlu
FR25	C	column_mapping_mode: 'source' (kolon aynen) ve 'mapping_file' (YAML mapping) desteği	P0 — Zorunlu
FR26	C	TriggerDagRunOperator ile Level1 → Level2 kademeli DAG tetikleme	P1 — Yüksek
FR27	C	validate_config() ile çalışma öncesi YAML doğrulama	P1 — Yüksek
FR28	C	source_type: 'script' ve load_method: 'script' ile hedef DB'de SQL script çalıştırma	P1 — Yüksek
FR29	C	sql_file parametresi ile harici .sql dosyasından sorgu okuma	P1 — Yüksek
FR30	C	Email Notification — hata durumunda Türkçe HTML email bildirimi (on_failure_callback)	P1 — Yüksek
FR31	C	ETL Studio: Şema/tablo keşfi (50 tablo limiti, typeahead arama), DAG güncelleme, tag yönetimi, timeline	P1 — Yüksek
FR32	C	Transformer TABLE_RULES: strip, parse_dates, to_numeric, empty_to_null, add_etl_date kuralları	P2 — Orta

Fonksiyonel Olmayan Gereksinimler (NFR)

ID	Gereksinim	Hedef Değer
NFR01	Bellek kullanımı (veri hacminden bağımsız)	< 500MB (Community), < 256MB (Enterprise)
NFR02	Throughput (batch aktarım, queue overhead olmadan)	100,000+ satır/saniye
NFR03	10M+ satır aktarım süresi	< 3 dakika (bulk modda)
NFR04	Eş zamanlı job yürütme (parallelism)	Airflow Dynamic Task Mapping
NFR05	Veri tutarlılık ve doğruluk oranı	%100
NFR06	Yeni Dialect ekleme süresi	< 1 hafta
NFR07	Güvenli erişim ve denetim kayıtları	Airflow log + JSON structured logging
NFR08	CI/CD test koşum süresi	Yeni Dialect eklenmesinde tüm integration test suite tetiklenir

Riskler & Varsayımlar

Risk	Olasılık	Etki	Önlem
Mevcut araçlardan geçiş maliyeti	Orta	Yüksek	Community versiyonu ücretsiz sunularak düşük geçiş bariyeri
Kurumsal güvenlik/uyumluluk beklentileri	Yüksek	Yüksek	7/24 SLA + Enterprise destek + RSA lisans modülü
C-extension bellek sızıntısı (Enterprise)	Orta	Kritik	Valgrind zorunlu CI/CD entegrasyonu
Proje kapsamının erken genişlemesi	Yüksek	Orta	CDC ve veri maskeleme kapsam dışı tutuldu
Veritabanı bağlantı timeout (uzun aktarım)	Orta	Yüksek	TCP keep-alive, connection pooler stratejisi